

Cenario RPG:

Personagem que Vira Zumbi

Blindando o invariante $HP \geq 0$ com Encapsulamento em Python



Problema

HP fica negativo sem restricao



Solucao

Encapsulamento + invariante



Resultado

Estado invalido impossivel

Diego Matheus | Murilo Alves | Vinicius Villalobos

Por que o personagem vira zumbi?

```
# Sem encapsulamento – estado exposto  
hp = 100 # variavel solta  
hp = hp - 9999 # dano absurdo  
print(hp) # resultado: -9899 🧟
```

Codigo sem encapsulamento

HP Negativo

Nenhuma regra impede que hp ultrapasse zero

Estado Exposto

Qualquer parte do codigo altera hp diretamente

Sem Invariante

Nao existe contrato que garanta $hp \geq 0$

Classe Personagem em Python

```
class Personagem:

    def __init__(self, nome, hp_inicial):

        if hp_inicial <= 0:

            raise ValueError("HP deve ser > zero")

        self.__nome = nome

        self.__hp = hp_inicial # privado

    def receber_dano(self, dano):

        if dano < 0:

            raise ValueError("Dano invalido")

        self.__hp = max(0, self.__hp - dano)

    def get_hp(self): return self.__hp

    def esta_vivo(self): return self.__hp > 0
```

Construtor valida

Atributo Privado (__hp)

Invariante: max(0, ...)

Getter publico

RESULTADO

Codigo funcionando — Teste de Resistencia

RPG.py U X

C:\> Atividades > 2026 > PARADIGMAS > Lab3 > RPG.py

```
1 class Personagem:
2     def __init__(self, nome, hp_inicial):
3         if hp_inicial <= 0:
4             raise ValueError("HP inicial deve ser maior que zero.")
5         self.__nome = nome
6         self.__hp = hp_inicial # atributo privado, nasce valido
7
8     def receber_dano(self, dano):
9         if dano < 0:
10             raise ValueError("Dano nao pode ser negativo.")
11         self.__hp = max(0, self.__hp - dano) # invariante: hp trava em zero
12
13     def get_hp(self):
14         return self.__hp
15
16     def get_nome(self):
17         return self.__nome
18
19     def esta_vivo(self):
20         return self.__hp > 0
21
22
23 def main():
24     heroi = Personagem("Diego", 100)
25
26     print(f"Personagem: {heroi.get_nome()} | HP: {heroi.get_hp()}")
27
28     heroi.receber_dano(30)
29     print(f"Apos 30 de dano | HP: {heroi.get_hp()}")
30
31     heroi.receber_dano(9999) # tentando deixar vida negativa
32     print(f"Apos dano de 9999 | HP: {heroi.get_hp()}")
33     print(f"Herói ainda vivo? {heroi.esta_vivo()}")
34
35     try:
36         heroi.receber_dano(-50) # tentando dano negativo
37     except ValueError as e:
38         print(f"BLOQUEADO: {e}")
39
40 main()
```

Windows PowerShell

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! <https://aka.ms/PSWindows>

PS C:\Atividades\2026\PARADIGMAS\Lab3> python RPG.py
Personagem: Diego | HP: 100
Apos 30 de dano | HP: 70
Apos dano de 9999 | HP: 0
Herói ainda vivo? False
BLOQUEADO: Dano nao pode ser negativo.
PS C:\Atividades\2026\PARADIGMAS\Lab3> |

Checklist de Engenharia OO

O objeto protege seu proprio estado?

Sim. `__hp` e privado e inacessivel diretamente do main.



Existe um Invariante claro?

Sim. `max(0, hp - dano)` garante `hp >= 0` em toda transicao.



Impossivel acessar variaveis direto do main?

Sim. Name mangling do Python bloqueia acesso a `__hp`.



Toda transicao de estado e validada?

Sim. `receber_dano` valida `dano` antes de qualquer alteracao.



Obrigado

Diego Matheus | Murilo Alves | Vinicius Villalobos

Paradigmas de Programacao • Lab 3 • 2026